Introdução ao Git

Gerenciamento de Repositórios e Pull Requests

Introdução ao Uso Avançado do GitHub

✅ **O que vamos aprender hoje:**

- Pull Requests e Forks 📤
- Code Review e boas práticas ✅
- Lidando com conflitos 🔀
- Integrando mudanças de maneira eficiente 🚀

O que é uma Pull Request (PR)?

🔹 **Uma Pull Request (PR)** é uma solicitação para mesclar mudanças de um branch para outro.

🔹 **Para que serve?**
✅ Permite colaboração em equipe.
✅ Facilita a revisão do código antes da integração.
✅ Ajuda a manter a qualidade do código.

🔹 **Fluxo de uma Pull Request:**
1️⃣ Você cria um branch e faz mudanças.
2️⃣ Abre uma Pull Request no GitHub.
3️⃣ Colegas revisam o código e sugerem melhorias.
4️⃣ Se aprovado, o código é mesclado no branch principal.

📌 **Dica:** PRs são essenciais no **GitFlow** e na colaboração com forks!

Criando um Pull Request (PR)

1️⃣ **Crie um branch para a alteração:**

```
git checkout -b minha-feature
```

2️⃣ **Faça commits e envie para o seu fork:**

```
git push origin minha-feature
```

3️⃣ **No GitHub, vá até o seu repositório e clique em “New Pull Request”**
4️⃣ **Escolha a branch correta e envie o PR**.

📌 **Dica:** Escreva uma boa descrição no PR explicando suas mudanças!

Code Review e Boas Práticas

🔹 **O que é Code Review?**

- Processo de revisar código antes de aprová-lo para o repositório principal.
- Ajuda a manter a qualidade do código e evitar bugs.

🔹 **Boas Práticas:**
✅ Escreva commits descritivos.
✅ Separe mudanças grandes em PRs menores.
✅ Utilize **comentários no PR** para discutir o código.
✅ Corrija feedbacks e atualize o PR:

```
git commit --amend

git push origin minha-feature --force
```

📌 **Dica:** No GitHub, use **"Request Changes"** se precisar que o autor faça ajustes!

Lidando com Conflitos no Git

- Conflitos ocorrem quando duas pessoas editam a mesma linha de um arquivo.
- Para resolver um conflito:

```
git pull origin main

git merge main
```

- O Git marcará o conflito no arquivo. Edite e escolha a versão correta.
- Depois, finalize com:

```
git add arquivo-conflito.txt

git commit -m "Resolve conflito"

git push origin minha-feature
```

📌 **Dica:** Ferramentas como VS Code e GitHub Desktop facilitam a resolução de conflitos.

O que é um Fork e quando usar?

- **Fork** é uma cópia de um repositório para a sua conta no GitHub.
- Permite contribuir com repositórios que você **não tem permissão de escrita**.
- Após o fork, você pode:
  - Criar branches, modificar o código e testar localmente.
  - Enviar um **Pull Request** para sugerir mudanças no projeto original.

📌 **Exemplo:** Fork de um projeto open-source para adicionar uma funcionalidade.

Como Criar um Fork e Clonar Localmente

1. No GitHub, clique em **Fork** no repositório desejado.
2. Clone o fork para sua máquina:

```
git clone https://github.com/seu-usuario/nome-do-repositorio.git
```

3. Adicione o repositório original como **remote upstream**:

```
git remote add upstream
https://github.com/original-usuario/nome-do-repositorio.git
```

4. Verifique os remotes configurados:

```
git remote -v
```

Mantendo o Fork Atualizado

- Antes de contribuir, mantenha seu fork atualizado com o repositório original:

```
git checkout main
git fetch upstream
git merge upstream/main
```

- Se quiser evitar merges, pode **rebasear**:

```
git rebase upstream/main
```

📌 **Dica:** Sempre atualize antes de iniciar uma nova feature para evitar conflitos!

Exercício Prático

1. Clone o repositório no GitHub: https://github.com/hamc/fatec20261
2. Crie uma branch chamada documentation/<meu_nome>
3. Crie o arquivo seu-nome-sobrenome.md e adicione uma linha com o texto “testando pull request “.
4. Faça um commit e envie para o Github.
5. Abra um **Pull Request** para o repositório
6. Simule um Code Review:
   - Comente no PR de um colega.
   - Sugira melhorias ou aprove o PR.
7. Se houver conflito, resolva e finalize o merge!

📌 **Objetivo:** Criar Pull Request, Code Review e resolução de conflitos

Dúvidas?